# YACI — Manual de Usuario

Hugo Coto

Este documento es el manual de usuario de **YACI** (*Yet Another Calculator Interpreter*). Incluye secciones de tutorial y de referencia. Tras leerlo deberías ser capaz de usar el lenguaje sin necesidad de consultar otras fuentes.

April 11, 2026

# Contents


1. Descripción ... 5
2. Algunos ejemplos simples ... 6
3. Compilación e instalación ... 7
4. El modo interactivo (REPL) ... 8
   4.1. Edición de línea ... 8
   4.2. Historial ... 8
   4.3. Completado con Tab ... 8
   4.4. Cambiar el prompt ... 8
5. Archivos de script ... 9
6. Números ... 10
   6.1. Caracteres ... 10
7. Texto ... 11
8. Variables ... 12
9. Constantes ... 13
   9.1. Constantes predefinidas ... 13
   9.2. Constantes de usuario ... 13
10. Operadores y expresiones ... 15
   10.1. Aritmética básica ... 15
   10.2. Precedencia ... 15
   10.3. Conversión de tipo (`as`) ... 16
   10.4. Alias textuales ... 16
   10.5. Aserción (`eq!`) ... 17
11. Listas ... 18
12. Valores booleanos ... 19
13. Silenciar la salida ... 20
14. Comentarios ... 21
15. Comandos del intérprete ... 22
   15.1. `clear` ... 22
   15.2. `quit` y `exit` ... 22
   15.3. `workspace` (`ws`) ... 22
   15.4. `del` / `delete` ... 22
   15.5. `verbose` ... 23
   15.6. `echo` ... 23
16. Cargar archivos ... 24
17. Llamadas a funciones externas (FFI) ... 25
   17.1. Enlazar una biblioteca ... 25
   17.2. Llamar una función ... 25
   17.3. Resumen de la sintaxis de llamada ... 26
18. Opciones de línea de comandos ... 27
19. Mensajes de error ... 28
20. Ejemplos completos ... 29
   20.1. Script de cálculo físico ... 29
   20.2. Llamada a la biblioteca matemática ... 29

20.3. Cargar constantes desde un archivo ..... 30
21. Apéndice — Gramática formal ..... 31
22. Glosario ..... 33

Este documento fue generado por IA. Es posible que contenga alguna incorrección.

# 1. Descripción

**YACI** es una herramienta para evaluar expresiones matemáticas y de texto de forma interactiva. Lee las expresiones que escribes, las evalúa, e imprime el resultado. También puede ejecutar archivos de script y llamar directamente a funciones de bibliotecas C ya compiladas.

El manual incluye las siguientes secciones:

**Algunos ejemplos simples** una primera toma de contacto con el intérprete.
**Compilación e instalación** cómo compilar YACI y sus dependencias.
**El modo interactivo (REPL)** edición de línea, historial y completado.
**Archivos de script** formato de los archivos `.yc`.
**Números** cómo escribir enteros, decimales y caracteres.
**Texto** cadenas de texto y sus secuencias de escape.
**Variables** guardar y reutilizar valores.
**Constantes** valores predefinidos y constantes de usuario.
**Operadores y expresiones** aritmética, precedencia y conversión de tipos.
**Listas** colecciones de valores.
**Valores booleanos** verdadero y falso como números.
**Silenciar la salida** el punto y coma y otras formas de suprimir resultados.
**Comentarios** cómo documentar tus scripts.
**Comandos del intérprete** `clear`, `quit`, `workspace`, `del`, `verbose`, `echo`.
**Cargar archivos** ejecutar scripts desde el intérprete.
**Llamadas a funciones externas** enlazar bibliotecas C y llamar sus funciones.
**Opciones de línea de comandos** cómo arrancar YACI con distintas configuraciones.
**Mensajes de error** errores frecuentes y cómo resolverlos.
**Ejemplos completos** scripts reales que combinan varias funcionalidades.
**Apéndice — Gramática formal** referencia completa de la sintaxis.
**Glosario** definiciones de los términos usados en este manual.

## 2. Algunos ejemplos simples

Para empezar, veamos lo que YACI hace con algunos ejemplos sencillos. Lo más básico es escribir un número y pulsar Intro:

```
>> 42
=42
```

El prefijo `=` indica que eso es el resultado de la expresión. Si escribes una operación aritmética, YACI la evalúa:

```
>> 2 + 3
=5
>> 10 / 4
=2.5
>> 2 ^ 8
=256
```

Puedes guardar resultados en una variable con `=`:

```
>> radio = 5
=5
>> pi * radio ^ 2
=78.5398
```

Y puedes usar cadenas de texto:

```
>> "hola"
="hola"
```

Eso es suficiente para empezar. En las secciones siguientes veremos cada concepto en detalle.

## 3. Compilación e instalación

YACI se compila con `make`. Necesita las bibliotecas **readline** y **libffi**:

```sh
# Ubuntu / Debian
apt install libreadline-dev libffi-dev

# macOS (Homebrew)
brew install readline libffi

# Compilar el intérprete
make

# Ejecutar las pruebas
make test
```

Una vez compilado, puedes arrancarlo con:

```sh
./yaci          # modo interactivo (REPL)
./yaci mi.yc    # ejecutar el archivo mi.yc y luego entrar al REPL
./yaci mi.yc --norepl   # solo ejecutar el archivo, sin REPL
```

Si estás aprendiendo el lenguaje, se recomienda usar la opción `--verbose` (o su atajo `-v`) para obtener sugerencias adicionales cuando se produzcan errores:

```sh
./yaci --verbose
```

# 4. El modo interactivo (REPL)

Cuando ejecutas `./yaci` sin argumentos (o sin `--norepl`), entras en el **REPL** (*Read-Eval-Print Loop*): el intérprete muestra el prompt `>>`, lee una línea, la evalúa e imprime el resultado.

## 4.1. Edición de línea

La edición de línea usa la biblioteca **readline** con el modo de edición **vi** activado por defecto. Esto significa que puedes usar comandos de movimiento de vi para editar la línea actual:

| Tecla | Acción |
|---|---|
| `←` / `→` | mover el cursor un carácter |
| `Ctrl-A` / `Ctrl-E` | ir al principio / final de la línea |
| `Ctrl-K` | borrar hasta el final de la línea |
| `Ctrl-U` | borrar la línea entera |
| `Backspace` | borrar el carácter anterior |
| `Ctrl-C` | cancelar la línea actual (sin salir) |

## 4.2. Historial

Cada línea que introduces se guarda en el historial de la sesión actual. Puedes navegar por él con las flechas arriba y abajo:

| Tecla | Acción |
|---|---|
| `↑` | línea anterior del historial |
| `↓` | línea siguiente del historial |

## 4.3. Completado con Tab

Pulsa `Tab` para completar comandos. Cuando el cursor está al principio de la línea, YACI completa con los comandos del intérprete (`exit`, `quit`, `load`, `clear`, `verbose`, `echo`). Cuando el cursor está en mitad de una ruta que comienza con `./` o `../`, el completado utiliza el sistema de archivos del shell.

```
>> lo<Tab>          →  load
>> ./sc<Tab>        →  ./scripts/
```

## 4.4. Cambiar el prompt

El prompt por defecto es `>>`. Puedes cambiarlo con la opción `--prompt`:

```sh
./yaci --prompt "yaci> "
```

# 5. Archivos de script

Un archivo de script de YACI es un archivo de texto plano con extensión `.yc`. Contiene las mismas expresiones y comandos que escribirías en el REPL, una por línea.

El intérprete procesa cada línea de arriba hacia abajo. Los resultados se imprimen (si `echo` está activo) o se suprimen con `;`. Las variables y constantes definidas en un script quedan en el espacio de trabajo al volver al REPL.

Ejemplo mínimo de script (`saludo.yc`):

```
# saludo.yc
saludo = "Hola, Mundo"
saludo
```

Ejecución:

```sh
$ ./yaci saludo.yc --norepl
=Hola, Mundo
```

Las líneas en blanco y los comentarios se ignoran. No hay límite en la profundidad de inclusión de archivos, pero el intérprete detecta bucles de carga circular y emite un error.

# 6. Números

El tipo numérico principal de YACI se llama `num` y almacena un número en coma flotante de doble precisión. Los literales se escriben de la forma habitual:

```
>> 42
=42
>> 3.14
=3.14
>> .5
=0.5
>> 2.9e-1
=0.29
>> 1e6
=1e+06
```

También están disponibles los valores especiales `NAN` (no es un número) e `INF` (infinito):

```
>> INF
=inf
>> NAN
=nan
```

## 6.1. Caracteres

Un carácter entre comillas simples es un literal entero ( `dec` ) cuyo valor es el código ASCII del carácter:

```
>> 'A'
=65
>> 'z'
=122
```

Nótese que `'A'` y `65` son el mismo valor. Este tipo resulta útil para pasar argumentos enteros a funciones externas (véase la sección sobre FFI).

## 7. Texto

Las cadenas de texto se delimitan con comillas dobles o con comillas simples. Ambas formas se comportan de manera idéntica: el carácter de cierre debe ser el mismo que el de apertura, y el otro tipo de comilla puede aparecer en el contenido sin necesidad de escaparla:

```
>> "hola mundo"
="hola mundo"
>> 'también funciona'
="también funciona"
>> "incluye 'comillas simples' sin problema"
="incluye 'comillas simples' sin problema"
>> 'incluye "comillas dobles" sin problema'
="incluye "comillas dobles" sin problema"
```

Dentro de una cadena se pueden usar las siguientes secuencias de escape:

| Secuencia | Significado |
|---|---|
| `\\` | barra invertida literal |
| `\'` | comilla simple |
| `\"` | comilla doble |
| `\n` | nueva línea |
| `\t` | tabulador |
| `\r` | retorno de carro |
| `\0` | carácter nulo |
| `\a` | alerta (pitido) |
| `\b` | retroceso |
| `\f` | avance de página |
| `\v` | tabulador vertical |

Una cadena sin cerrar al final de la línea provoca un error:

```
>> "sin cerrar
Error: Unclosed string "sin cerrar
```

**Nota:** Los operadores aritméticos (`+`, `-`, `*`, `/`, `^`) **no** se pueden aplicar a cadenas. Para combinar texto y números en la salida, imprime cada valor en una línea separada.

# 8. Variables

Para guardar un valor y reutilizarlo, usa el operador de asignación `=`:

```
>> x = 10
=10
>> x + 5
=15
>> x
=10
```

Los nombres de variable pueden contener letras, dígitos y guiones bajos `_`, pero deben **comenzar** por una letra o por `_`:

```
>> resultado = 0
=0
>> _tmp = 1
=1
>> valor2 = 2
=2
```

La asignación es en sí misma una expresión y devuelve el valor asignado, así que puedes usarla dentro de otras expresiones o encadenarlas:

```
>> a = b = 7
=7
>> a
=7
>> b
=7
```

Si intentas usar una variable antes de asignarle un valor, obtendrás un error:

```
>> z
Error: Variable `z` not defined
Hint: Maybe you want to type `z = 1`
```

# 9. Constantes

## 9.1. Constantes predefinidas

El intérprete carga automáticamente las siguientes constantes matemáticas de la biblioteca estándar de C. Son de solo lectura y siempre están disponibles:

| Nombre | Valor aprox. | Descripción |
|---|---|---|
| `e` | 2.71828 | Número de Euler |
| `log2e` | 1.44269 | $\log_2(e)$ |
| `log10e` | 0.43429 | $\log_{10}(e)$ |
| `ln2` | 0.69315 | $\ln(2)$ |
| `ln10` | 2.30259 | $\ln(10)$ |
| `pi` | 3.14159 | $\pi$ |
| `pi_2` | 1.57080 | $\pi$ / 2 |
| `pi_4` | 0.78540 | $\pi$ / 4 |
| `sqrt2` | 1.41421 | $\sqrt{2}$ |

También están disponibles los enteros `zero`, `one`, `two`, ..., `ten` (del 0 al 10).

```
>> pi
=3.14159
>> e ^ 2
=7.38906
>> one + two
=3
```

## 9.2. Constantes de usuario

Puedes definir tus propias constantes con la palabra clave `const`. Una vez asignadas, no se pueden modificar:

```
>> const GRAVEDAD = 9.8
=9.8
>> GRAVEDAD = 10
Error: Assigning to a constant var
```

Las constantes de usuario se comportan igual que las variables, salvo que intentar reasignarlas es un error. Esto es útil para valores que no deben cambiar accidentalmente a lo largo de un script.

Si necesitas redefinir una constante, primero debes eliminarla con `del` (véase la sección `del`):

```
>> const FACTOR = 2
=2
```

```
>> del FACTOR
>> FACTOR = 3
=3
```

# 10. Operadores y expresiones

## 10.1. Aritmética básica

YACI admite los cinco operadores aritméticos habituales:

```
>> 2 + 3
=5
>> 10 - 4
=6
>> 3 * 7
=21
>> 9 / 4
=2.25
>> 2 ^ 10
=1024
```

La negación unaria también está disponible:

```
>> -5
=-5
>> -(3 + 2)
=-5
```

## 10.2. Precedencia

Las reglas de precedencia son las habituales en matemáticas. De mayor a menor precedencia:

1. Conversión de tipo ( `as` )
2. Potenciación ( `^` , asociativa por la derecha)
3. Negación unaria ( `-` )
4. Multiplicación y división ( `*` , `/` )
5. Suma y resta ( `+` , `-` )
6. Aserción ( `eq!` )
7. Asignación ( `=` )

Los paréntesis alteran el orden de evaluación:

```
>> 2 + 3 * 4
=14
>> (2 + 3) * 4
=20
>> 2 ^ 3 ^ 2
=512
>> (2 ^ 3) ^ 2
=64
```

## 10.3. Conversión de tipo (`as`)

El operador `as` convierte un valor de un tipo a otro. Es útil, por ejemplo, para pasar enteros a funciones que los requieren, o para convertir un número a texto:

```
>> 3.14 as str
="3.14"
>> "12.5" as num
=12.5
>> 1 as int
=1
>> 1 as list
={1}
>> {2} as num
=2
```

Los nombres de tipo que acepta `as` son:

| Nombre | Alias | Tipo resultante |
|---|---|---|
| `num` | `float` | Coma flotante de doble precisión |
| `dec` | `int` | Entero de 32 bits |
| `str` | | Cadena de texto |
| `list` | | Lista de un elemento |

Las conversiones entre `num` y `dec` son siempre válidas. Convertir una lista a `num` o `str` solo funciona si la lista tiene exactamente un elemento.

## 10.4. Alias textuales

Los operadores `+` y `-` tienen alias en inglés para quienes prefieren una escritura más natural:

```
>> 2 plus 3
=5
>> 10 minus 4
=6
```

De la misma forma, la asignación acepta la sintaxis `let` … `be` … :

```
>> let velocidad be 9.8
=9.8
>> let tiempo be 3
=3
>> velocidad * tiempo
=29.4
```

`let` se ignora silenciosamente; `be` actúa exactamente igual que `=`.

## 10.5. Aserción (`eq!`)

El operador `eq!` comprueba que dos expresiones tienen el mismo valor. Si coinciden, devuelve el valor sin hacer nada más. Si **no** coinciden, imprime ambos lados y **termina el programa** con un error. Es útil para escribir pruebas automáticas dentro de scripts:

```
# en un script de pruebas
1 + 1 eq! 2
pi * 2 eq! 6.28318
"hola" as list eq! {"hola"}
```

Si la aserción falla verás algo como:

```
Error: Assertion error:
  lvalue: 3
  rvalue: 4
Error: Values doesn't match
```

# 11. Listas

Una lista es una secuencia ordenada de valores. Se escribe entre llaves con los elementos separados por comas:

```
>> {1, 2, 3}
={1, 2, 3}
>> {"a", "b"}
={"a", "b"}
>> {}
={}
```

Las listas se usan principalmente para pasar varios argumentos a una función (véase la sección FFI). También se puede convertir un valor a lista con `as list`, y extraer el elemento de una lista de un elemento con `as num`, `as int` o `as str`:

```
>> 42 as list
={42}
>> {42} as num
=42
>> {"hola"} as str
="hola"
```

## 12. Valores booleanos

YACI no tiene un tipo booleano independiente. Los valores verdadero y falso son simplemente números: `1` para verdadero y `0` para falso. Sin embargo, el intérprete acepta varias grafías que se convierten automáticamente:

| Verdadero (→ 1) | Falso (→ 0) |
|---|---|
| `true`, `True`, `TRUE` | `false`, `False`, `FALSE` |
| `on`, `On`, `ON` | `off`, `Off`, `OFF` |
| `yes`, `Yes`, `YES` | `no`, `No`, `NO` |

Estas palabras también se usan como argumentos de los comandos `verbose` y `echo` (`verbose true`, `echo off`, etc.).

## 13. Silenciar la salida

Por defecto, el resultado de cada expresión se imprime en la pantalla. Para que una expresión **no** imprima su resultado, añade `;` al final:

```
>> a = 10;
>> a
=10
```

La primera línea asigna `a = 10` pero no muestra nada. La segunda imprime el valor normalmente.

El punto y coma también funciona solo, como una expresión vacía:

```
>> ;
```

También existe la forma `shh!` como alternativa compacta al punto y coma:

```
>> a = 10 shh!
>> a
=10
```

Por último, hay dos variantes en inglés natural pensadas para que los scripts se lean como prosa:

```
a = 10, but don't show any output
a = 10, but be quiet
```

Todas estas formas son exactamente equivalentes a `a = 10;`.

## 14. Comentarios

Los comentarios de línea se introducen con `#` o con `//`. El intérprete ignora todo lo que hay desde el marcador hasta el final de la línea:

```
# Esto es un comentario
// Esto también lo es

radio = 5          # el radio del círculo
area = pi * radio ^ 2   // el área
```

Ambas formas son equivalentes y pueden usarse indistintamente. No existe sintaxis de comentario de bloque multilínea.

# 15. Comandos del intérprete

Los comandos son instrucciones especiales que controlan el comportamiento del intérprete. Se escriben solos en una línea y **no** llevan punto y coma. La mayoría no distinguen mayúsculas de minúsculas.

## 15.1. `clear`

Borra la pantalla del terminal:

```
clear
```

## 15.2. `quit` y `exit`

Termina el intérprete:

```
quit
exit
```

Ambas formas son equivalentes.

## 15.3. `workspace` (`ws`)

Muestra las variables que están definidas en este momento. Se puede abreviar como `ws`. Con un argumento filtra por nombre, tipo o archivo de origen:

```
workspace              # todas las variables
ws                     # ídem, forma abreviada
workspace x            # solo la variable 'x'
workspace num          # solo las de tipo num
workspace ./mi.yc      # solo las definidas en ese archivo
```

Esto es útil para inspeccionar el estado del intérprete en cualquier momento.

## 15.4. `del` / `delete`

Elimina una variable del espacio de trabajo. Esto permite reutilizar el nombre, incluso si era una constante. Puede escribirse como `del` o `delete`:

```
del x              # elimina la variable 'x'
delete x           # ídem
del all            # elimina todas las variables
del ./mi.yc        # elimina las definidas en ese archivo
```

Por ejemplo, si defines una constante en un script y luego la quieres redefinir, puedes eliminarla con `del` antes de la nueva asignación:

```
>> const FACTOR = 2
=2
>> del FACTOR
>> FACTOR = 3
```

```
=3
```

### 15.5. `verbose`

Activa o desactiva las sugerencias (*hints*) que aparecen junto a los mensajes de error. Es especialmente útil cuando estás aprendiendo el lenguaje:

```
verbose on
verbose off
```

También puedes arrancarlo con `-v` desde la línea de comandos:

```sh
./yaci --verbose
```

Con `verbose on`, un error como «paréntesis sin cerrar» incluirá una sugerencia adicional que te indica exactamente cómo corregirlo:

```
>> f(1, 2
Error: Unclosed parenthesis
Hint: Don't forget to close it as in `f(...)`
```

### 15.6. `echo`

Activa o desactiva la impresión automática del resultado de cada expresión. Con `echo off` el intérprete solo imprime lo que tú le pides explícitamente:

```
>> echo off
>> 1 + 1
>> echo on
>> 1 + 1
=2
```

## 16. Cargar archivos

Puedes guardar expresiones en un archivo de texto con extensión `.yc` y cargarlo desde el intérprete con `load`. Las variables que defina el archivo quedan disponibles en el espacio de trabajo:

```
# archivo constantes.yc
const TAX = 0.21;
const BASE = 100;
```

```
>> load ./constantes.yc
>> BASE * (1 + TAX)
=121
```

Las rutas de archivo se pueden escribir de tres formas:

```
load ./scripts/mi_script.yc   # ruta relativa con ./
load <scripts/mi_script.yc>   # entre ángulos
load scripts/mi_script.yc     # solo si tiene extensión
```

Una ruta también se puede guardar en una variable y cargarse después:

```
>> archivo = <scripts/mi_script.yc>
>> load archivo
```

Los archivos cargados pueden a su vez cargar otros archivos. El intérprete detecta bucles de carga y emite un error antes de que ocurra un desbordamiento de pila:

```
Error: Recursion detected on load
Hint: Check that there isn't a load-loop
```

# 17. Llamadas a funciones externas (FFI)

YACI puede llamar a funciones de cualquier biblioteca compartida (`.so` en Linux, `.dylib` en macOS). El proceso es: primero enlazar la biblioteca con `open` (o su alias `link`), y luego llamar la función como si fuera nativa.

## 17.1. Enlazar una biblioteca

```
open ./mi_biblioteca.so       # biblioteca local (también: link)
link ./mi_biblioteca.so       # ídem, alias de open
open <math.h>                 # alias para libm (solo GNU/Linux)
```

El alias `<math.h>` es una forma cómoda de enlazar la biblioteca matemática del sistema en GNU/Linux sin necesidad de conocer su nombre real.

Una vez enlazada, cualquier función de esa biblioteca queda disponible para ser llamada.

## 17.2. Llamar una función

La sintaxis básica es igual que en la mayoría de los lenguajes:

```
nombre_funcion(arg1, arg2, ...)
```

Por ejemplo, usando la biblioteca matemática:

```
>> open <math.h>
>> log(e)
=1
>> log(e ^ 2)
=2
>> sqrt(2)
=1.41421
```

Por defecto YACI asume que la función devuelve un `num` (coma flotante). Si la función devuelve un entero o una cadena, hay que indicarlo con `->`:

```
>> strcspn("012345", "5") -> int
=5
>> strdup("hola") -> str
="hola"
```

Los argumentos enteros se obtienen convirtiendo con `as int`. Esto es necesario cuando la función C espera un `int` en lugar de un `double`:

```
>> open ./libraylib.so
>> InitWindow(800 as int, 450 as int, "Mi ventana")
```

## 17.3. Resumen de la sintaxis de llamada

```
funcion(args)              # retorno num por defecto
funcion(args) -> num       # retorno num (explícito)
funcion(args) -> int       # retorno dec (entero)
funcion(args) -> str       # retorno str (cadena)
```

# 18. Opciones de línea de comandos

```sh
./yaci [opciones] [archivo ...]
```

| Opción | Atajo | Efecto |
|---|---|---|
| `--norepl` | | No entrar al modo REPL después de procesar los archivos. |
| `--verbose` | `-v` | Activar sugerencias en errores desde el inicio. |
| `--noecho` | `-E` | No imprimir el resultado de cada expresión. |
| `--nocolor` | `-C` | Desactivar los colores en la salida. |
| `--prompt X` | | Usar `X` como prompt del REPL (por defecto `>>` ). |
| `--help` | `-h` | Mostrar el mensaje de ayuda y salir. |

Ejemplo de uso habitual para ejecutar un script sin entrar al REPL:

```sh
./yaci mi_script.yc --norepl
```

Múltiples archivos se procesan en orden:

```sh
./yaci base.yc extensiones.yc principal.yc --norepl
```

# 19. Mensajes de error

Cuando el intérprete encuentra un problema, imprime un mensaje de error en rojo (si los colores están activados). Con `verbose on` o `-v` también imprime una sugerencia (*hint*) en verde.

Los errores más frecuentes son:

| Error | Cuándo ocurre |
|---|---|
| `Variable x not defined` | Se usa `x` sin haberla asignado antes. |
| `Assigning to a constant var` | Se intenta modificar una constante. |
| `Can't change var signature` | Se intenta redefinir con `const` una variable ya asignada. |
| `Unclosed parenthesis` | Se abre `(` pero no se cierra antes del fin de línea. |
| `Unclosed string` | Se abre `"` o `'` pero no se cierra antes del fin de línea. |
| `Malformed number` | El exponente de notación científica está mal formado (p.ej. `1e++2`). |
| `Couldn't find any function with this signature` | La función no existe en ninguna biblioteca enlazada. |
| `Calling a non-callable var` | Se intenta llamar como función a una variable que no lo es. |
| `Recursion detected on load` | Un archivo intenta cargarse a sí mismo directa o indirectamente. |
| `del expects an action` | `del` se escribe solo, sin indicar qué eliminar. |
| `Assign operation needs a value` | El operador `=` aparece al final de la línea sin valor en el lado derecho. |

Cuando `verbose` está activo, casi todos los errores van acompañados de una sugerencia (*hint*) que indica la causa probable y cómo corregirla.

# 20. Ejemplos completos

## 20.1. Script de cálculo físico

El siguiente ejemplo muestra variables, constantes y aritmética en un script real. Guárdalo como `fisica.yc`:

```
# fisica.yc — cálculo de caída libre

const g = 9.8;      # aceleración de la gravedad (m/s²)

t = 3;              # tiempo en segundos
v = g * t;          # velocidad final (m/s)
d = g * t ^ 2 / 2;  # distancia recorrida (m)

v
d
```

Ejecución:

```sh
$ ./yaci fisica.yc --norepl
=29.4
=44.1
```

## 20.2. Llamada a la biblioteca matemática

```
# matematicas.yc

open <math.h>

log(e)
log(e ^ 2)
log(e ^ (1/2))

# convertir de grados a radianes y calcular seno
grados = 90;
rad = grados * pi / 180;
sin(rad)
```

Ejecución:

```sh
$ ./yaci matematicas.yc --norepl
=1
=2
=0.5
=1
```

## 20.3. Cargar constantes desde un archivo

```
# unidades.yc
const KM_POR_MILLA = 1.60934;
const KG_POR_LIBRA = 0.453592;
```

```
>> load ./unidades.yc
>> 5 * KM_POR_MILLA
=8.0467
>> 150 * KG_POR_LIBRA
=68.0388
```

# 21. Apéndice — Gramática formal

Esta sección describe la gramática de YACI en notación BNF extendida. Los símbolos entre `<...>` son no terminales; los literales entre comillas son terminales; `|` separa alternativas; `[...]` indica opcionalidad.

```
programa     ::= { línea }

línea        ::= expr '\n'
               | expr ';' '\n'
               | ';' '\n'
               | '\n'
               | cmd '\n'

cmd          ::= 'clear'
               | 'quit' | 'exit'
               | 'workspace' [ VAR | TYPE | PATH | STR ]
               | 'ws'        [ VAR | TYPE | PATH | STR ]
               | 'del' ( 'all' | VAR | PATH | STR )
               | 'delete' ( 'all' | VAR | PATH | STR )
               | 'load' ( PATH | VAR )
               | 'open' ( PATH | VAR )
               | 'link' ( PATH | VAR )
               | 'verbose' BOOL
               | 'echo'    BOOL

expr         ::= NUM
               | DEC
               | STR
               | PATH
               | VAR
               | BOOL
               | '{' lista '}'
               | expr 'as' TYPE
               | expr 'eq!' expr
               | VAR '=' expr
               | 'const' VAR '=' expr
               | VAR '(' lista ')' [ '->' TYPE ]
               | expr ('+' | '-' | '*' | '/' | '^') expr
               | '-' expr
               | '(' expr ')'

lista        ::= ε | expr { ',' expr }

NUM          ::= dígito+ [ '.' dígito* ] [ ('e'|'E') ['+'|'-'] dígito+ ]
```

```
                | '.' dígito+
                | 'NAN' | 'INF'
DEC           ::= ''' carácter '''
STR           ::= '"' { carácterDQ } '"'
                | ''' { carácterSQ } '''
PATH          ::= './' ruta | '../' ruta | '<' ruta '>'
                | ruta '.' extensión
VAR           ::= (letra | '_') { letra | dígito | '_' }
TYPE          ::= 'num' | 'float' | 'dec' | 'int' | 'str' | 'list'
BOOL          ::= 'true'|'True'|'TRUE'|'yes'|'Yes'|'YES'|'on'|'On'|'ON'
                | 'false'|'False'|'FALSE'|'no'|'No'|'NO'|'off'|'Off'|'OFF'
SILENCIO      ::= ';'
                | 'shh!'
                | ', but don\'t show any output'
                | ', but be quiet'
COMENTARIO    ::= ('#' | '//') { carácter } '\n'
```

Las palabras reservadas (`as`, `eq!`, `const`, `let`, `be`, `plus`, `minus`, `all`, `true`, `false`, etc.) no pueden usarse como nombres de variable.

# 22. Glosario

**REPL** *Read-Eval-Print Loop*. Modo interactivo en el que el intérprete lee una línea, la evalúa e imprime el resultado, repitiendo el ciclo.

**num** Tipo numérico principal de YACI. Almacena un número en coma flotante de doble precisión (equivalente a `double` en C).

**dec** Tipo entero de YACI. Almacena un entero de 32 bits (equivalente a `int` en C). Se obtiene con literales de carácter ( `'A'` ) o con `as int`.

**str** Tipo cadena de texto de YACI. Secuencia de caracteres delimitada por comillas simples o dobles.

**list** Tipo colección de YACI. Secuencia ordenada de valores de cualquier tipo, delimitada por llaves ( `{...}` ).

**FFI** *Foreign Function Interface*. Mecanismo que permite llamar funciones compiladas en C desde YACI, sin necesidad de recompilar el intérprete.

**espacio de trabajo (*workspace*)** Conjunto de variables y constantes definidas en la sesión actual. Se puede inspeccionar con el comando `workspace` y vaciar con `del all`.

**eq!** Operador de aserción. Comprueba que dos valores son iguales y termina el programa con un mensaje de error si no lo son. Diseñado para escritura de pruebas en scripts.

**hint** Sugerencia que el intérprete imprime junto a ciertos mensajes de error cuando `verbose` está activo, indicando la causa probable y la corrección recomendada.

**script** Archivo de texto con extensión `.yc` que contiene expresiones y comandos YACI, procesado de arriba hacia abajo.